Assignment 0: mosinek

Repository: MOD550, and it is public
Delivered: 02.09.2026 17:32, two days early
README: there, but it does not say who you are. Add your name and student mail (no points lost)

87.5/100 + 23.75 bonus = 111.25 A

Task 0: (requirement)

Your repository is public. The assignment asks for a private one shared with me. No points are attached, switch the setting anyway.

Task 1: (10/10 pts)

ok

Task 2: (42.5/50 pts)

In `tools.py` (11.25/15 pt)

`for i,j in enumerate(lists)` then `for k in j`. Three single letters where i is a counter, j is a list and k is an element. In the dictionary version you wrote key and value and it reads immediately, so you know how to do it.

In `code.py` (15/15 pt)

ok

In `test_code.py` (11.25/15 pt)

The tests check types, not answers. `isinstance(num_sum, float)` passes whether the function adds correctly, adds twice or returns 0.0. The cause is upstream: you generate the test data randomly, so you cannot know the expected value. Tests want fixed data. Write four numbers you can add in your head, assert the sum, and keep the random generation for `code.py` where it belongs.

In `jupyther_code.ipynb` (5/5 pt)

ok

Task 3: (35/40 pts)

Folder and structure (10/10 pts)

ok

In `tools.py` (7.5/10 pts)

`if j > temperature[-1]` compares every reading against the last element of the whole list, not against the previous reading. Line 37 wants temperature[i-1]. As written the first condition is meaningless, since it asks whether each value beats the final one. Second: total_power_use

returns a cumulative list and `sum(cumulative_power)`, which adds up running totals and is not the total power used. The task asks for a function taking the list of instantaneous values and returning one number, and that number is sum(power_list).

In `code.py` (7.5/10 pts)

The total is never printed. You unpack cumul_sum on line 14 and then never use it, so the screen shows the plot and nothing else. The task asks for the total on screen too.

In `test_code.py` (10/10 pts)

ok. Checking that every power value is between 0 and 3 and that the cumulative series never decreases is the right way to test a function fed with random data, and it is what part a should have done.

Bonus: (+23.75)

- test with pytest: +5 (both folders collected and passing)
- use classes: +5
- numpy to generate the data: +5
- submit 1 day earlier: +5 (last commit 02.09 17:32)
- pylint 10/10: +3.75 (10.00 in project_1_b and 9.78 in project_1_a, close enough for three quarters)

Overall

Every bonus except the last quarter of pylint, and the structure of the code is genuinely good: static methods on a class, numpy generated inputs, docstrings that state the assumptions before the rules.

The three things that cost you are all in the same family. `temperature[-1]` instead of `temperature[i-1]`, a total function that returns a running series instead of a number, and a total that is computed and then never printed. None of them raise an error, which is exactly why they survived to me. When a function cannot fail loudly, test it against numbers you worked out by hand.

Writing the cumulative test as "never decreasing" rather than as a fixed expected list was your own idea and it is the right one.